Augmented Ontological Semantic Platform (A-OSP)

A browser-native epistemic operating environment for reconstructable, diagnosable, portable and governable AI-assisted work

Gianluca Conte
Independent Researcher
contegianluca@hotmail.com

May 2026

DOCUMENT STATUS

Experimental MVP-hardening whitepaper. This document describes an architecture, current repository evidence, hardening targets and measurable validation paths. It does not claim legal certification, production readiness or completed proof-grade implementation across all surfaces.

CORE THESIS

A-OSP uses the enterprise browser runtime as a standardized substrate and builds on top of it a text-first epistemic operating layer: typed stripes, schema registry, scoped query language, bounded model calls, proof contracts, error-truth envelopes, authority matrices, generated control surfaces, app lenses and guard/test architecture.

Field	Value
Prepared for	CTO, engineering, enterprise architecture, risk/governance and strategic review
Primary milestone	Proof-grade D1 and EQL, followed by artifact witness, Error Truth and enterprise validation
Theoretical reference	Public RLA-ECNN repository: https://github.com/Luke883i/ROA
Version date	2026-05-25

All maturity claims are tagged as AS-IS, HARDENING, TARGET, HYPOTHESIS or NON-GOAL.

Table of Contents

PART I - PROBLEM, THESIS AND MOAT DISCIPLINE

Why A-OSP exists, what it is, and how to read its claims.

1. Executive Summary

AI output is abundant. Governable AI-assisted work is still scarce. Most organizations can now generate text, summaries, code, analyses and reports, but they still struggle to reconstruct how those outputs were produced, which evidence supports them, what was inferred, what is missing, which transformations occurred, and which human review steps remain required. A-OSP addresses that gap by building a local, browser-native, text-first proof-and-knowledge substrate for AI-assisted work. Its durable source of truth is not the model, the user interface, the database or the export file. It is a structured local substrate of typed append-only stripes, readback checks, receipts, proof contracts, governance states and diagnostic events.

Core moat-code thesis
A-OSP's defensibility is not a single model or algorithm. It is the integration of typed stripes, schema registries, epistemic query/orchestration languages, proof contracts, error-truth envelopes, authority matrices, generated control surfaces and app lenses that make AI-assisted work queryable, inspectable, portable, diagnosable and governable.

Plain-language architecture

```
AI-assisted work
  -> local source object
  -> readback / receipt
  -> scoped retrieval
  -> bounded model transformation
  -> reviewable artifact
  -> human governance
```

The near-term objective is intentionally narrow: proof-grade D1 and EQL. D1 intake and EQL retrieval must become receipt-backed, readback-confirmed, observable and resistant to false-green UI states. The broader ambition is a browser-native epistemic operating environment in which humans can capture evidence, structure knowledge, bound model calls, inspect proof state, diagnose failures and govern outputs without surrendering source-of-truth to a model provider or SaaS workflow.

The current coded state of this milestone is explicitly bounded. D1/EQL proof-grade is not yet claimed as complete: the repository has landed proof contracts, DataIntake UI honesty and EQL L0-L2 foundations, while the D1 proof producer, persisted D1Receipt, EQLQueryReceipt, unified EQL proof route and C1 convergence artifacts remain hardening targets. See Appendix P - D1/EQL Proof-Grade Engineering Attestation.

Primitive	Meaning	Why it matters
Text-first source of truth	Durable epistemic state lives in .aosp.txt stripes.	Portable, diffable, recoverable, provider-independent.
Knowledge reification	Work becomes typed entities and relations.	Implicit work becomes queryable and governable.
Bounded model calls	LLMs operate under scoped refs, schemas and forbidden actions.	Model is processor, not owner of truth.
Operational proof	Proof is append, readback, receipt, lineage, witness, review.	Prevents UI or export from masquerading as proof.
Moat-custom layer	EQL, FAE, MONL, Schema Registry, Error Truth, Authority Matrix.	Turns architecture into inspectable control surfaces.

2. The Enterprise AI Proof Gap

The bottleneck is no longer generation. The bottleneck is proof, review, portability and accountability. A prompt-first workflow can generate an impressive report while leaving the organization unable to explain which evidence was used, which claims were unsupported, whether the result was persisted, whether it can be read back, and whether any human reviewer accepted the output.

Forbidden equivalences
output ≠ proof, confidence ≠ evidence, log ≠ receipt, export ≠ witness, UI green ≠ proof, fallback ≠ proof mode, model memory ≠ source of truth, review ≠ approval, capability unavailable ≠ proof failure, generated document ≠ authority source

Prompt-first AI	A-OSP pattern
fluent output	typed epistemic event
chat memory	text-first source of truth
prompt context	scoped reticular context
model answer	validated candidate object
export	artifact + witness
manual trust	readback + receipt + review

vendor memory	provider-replaceable processing

3. What A-OSP Is and Is Not

A-OSP is	A-OSP is not
browser-native epistemic operating environment	a native operating system
text-first virtual epistemic filesystem	arbitrary folder storage
proof-and-knowledge substrate	a chatbot or document generator
bounded model-call orchestration layer	model-provider memory
human-facing diagnostic workstation	autonomous governance engine
enterprise validation platform for proof-sensitive workflows	legal advice or compliance certification

Why "Ontological Semantic Platform"
A-OSP is ontological in the engineering sense: it turns work into typed entities and relations - answers, atoms, evidence gaps, enrichments, links, scores, kernels, artifacts, receipts, guards, errors and review states.
It is semantic because those entities are not opaque text blobs. They carry meaning through prefixes, schemas, relationships, source references, lineage, proof state, authority status and governance status.
It is augmented because bounded model calls can assist extraction, enrichment, synthesis and review while the source of truth remains outside the model.

4. Claims, Evidence, Maturity and Epistemic States

Tag	Meaning	Document rule
AS-IS	Present as documented architecture, implementation surface, repository evidence or working pattern.	Use when the claim is already visible in code/docs/audits.
HARDENING	Partially present, but being strengthened by current issue-chain or audit findings.	Use for proof wiring, UI honesty, validation or drift controls under completion.
TARGET	Expected after proof-convergence roadmap.	Use for planned proof objects or full orchestration not yet claimed.
HYPOTHESIS	Testable assumption, not proven.	Use for economics, scalability and performance claims.
NON-GOAL	Explicitly outside scope.	Use to prevent overclaim.

Operational epistemic states
knows -> evidence exists inside declared boundary
does not know -> no sufficient evidence is available
uncertain -> evidence is incomplete, contradictory, stale or below threshold
out of scope -> request exceeds ontology, permission or proof boundary
derived -> evidence comes from generated or secondary sources
stale -> superseded by newer or more authoritative source

These states should appear consistently in UI, EQL, ArtifactWitness, GovernanceRun, Observatory and Error Truth. They are the bridge between the theoretical epistemic stance and the actual product semantics.

PART II - BROWSER-NATIVE SUBSTRATE AND SEMANTIC FILESYSTEM

The browser is the runtime substrate; the filesystem and schema registry are the semantic substrate.

5. Browser-Native Epistemic Operating Environment

A-OSP is browser-native by design. The browser is not treated as a temporary presentation layer. It is treated as a standardized enterprise runtime substrate that already provides rendering, sandboxing, origin isolation, identity/session integration, permission boundaries, network transport, local storage capabilities, deployment/update channels and policy-managed execution.

Browser substrate stack

```
Enterprise browser runtime
  provides:
    rendering / sandboxing / origin isolation
    identity and session boundaries
    permission-mediated capabilities
    HTTP / WebSocket transport
    local storage and selected local file capabilities
    PWA / deployment / update channel

A-OSP epistemic operating layer
  builds:
    virtual text-first filesystem
    typed epistemic object model
    EQL / FAE / MONL moat-code surfaces
    proof contracts and Error Truth
    Observatory and app lenses
    guard/test control plane
```

Failure mode	Design response
Database-first hidden truth	.aosp.txt source-of-truth
API-centric complexity	filesystem-first access layer
Hybrid DB/files drift	source/cache/view/adapter/export/proof taxonomy
AI output mistaken for proof	receipt / witness / proof boundary
UI state mistaken for truth	Error Truth + Observatory
Agentic development drift	PR / issue / guard / authority matrix architecture
Generated docs mistaken for authority	Authority Matrix + generated control plane

6. Webapp Infrastructure and Runtime Topology

This chapter makes the software infrastructure of the A-OSP webapp explicit. The preceding chapter explains why the browser is the enterprise runtime substrate; this chapter describes how the webapp is composed, how data and proof-relevant state move through the system, and which controls make the runtime inspectable for CTO, investor, governance and government-grade review.

The goal is not to claim production readiness. The goal is to expose the runtime topology, responsibility boundaries and validation surfaces that must be hardened for enterprise pilots.

Runtime topology overview

Layer	Main components	Runtime role	Control / source of truth
Browser runtime	Enterprise browser sandbox, origin isolation, permissions, local storage, HTTP/WebSocket.	Standardized execution substrate; not a native OS.	Browser policy, capability state, security posture.
Frontend shell	React/Vite app shell, routes, app registry, window/app lens surfaces.	Human operating workspace over epistemic primitives.	App Support Tier, topology registry, E2E apps tests.
State layer	Hooks, app context, React Query cache, local UI state.	Coordinates UI state without owning durable truth.	Dataflow contract, projection model, stale-state checks.
Data access layer	useFilesystem, DataStore, queryWithStatus, readSourcePolicy.	Filesystem-first read/write discipline.	D-030, storage SSOT lint, read source policy guard.
Backend control plane	parser-service APIs, EQL, MONL, stripe operations, diagnostics.	Runtime control plane for epistemic operations.	API topology, health checks, telemetry, CI guards.

Cache/admin layer	Directus/PostgreSQL, auth/admin/index/cache functions.	Queryable cache and administration; not durable truth.	Cache rebuild tests, source/cache taxonomy.
Durable source	.aosp.txt stripes, schema registry, append/readback semantics.	Canonical epistemic state.	Stripe Schema Registry, invariants, backup/restore.
Realtime/diagnostics	WebSocket, Observatory, Error Truth, AOSPErrorEnvelope.	Runtime visibility for errors, proof state and capability state.	ErrorTruthBundle, MTTD/MTTR, fallback visibility.
AI adapter layer	Provider adapters, bounded calls, FAE profiles, provider exposure boundary.	Model as replaceable processor.	ProviderExposureBundle, schema validation, human review.
Operations/control	CI/CD, tests, guards, generated docs, authority/topology/security checks.	Keeps architecture from drifting.	OperationalMoatBundle, InvariantBundle, E2EDeterminismBundle.

End-to-end request, data and proof flow

```
Browser / React app
 -> App Shell + App Registry + App Lenses
 -> State Layer / Hooks / React Query
 -> useFilesystem / DataStore / queryWithStatus
 -> parser-service API
    -> EQL scoped retrieval
    -> MONL parsed-stripe operators
    -> stripe read/write and readback
    -> FAE bounded model orchestration
    -> LLM provider adapter
    -> telemetry / errors / proof state
 -> .aosp.txt durable source
 -> Directus/PostgreSQL cache-index-auth-admin
 -> Observatory / WebSocket / Error Truth
 -> Artifact / GovernanceRun / Proof Spine targets
```

Infrastructure responsibility boundaries

Boundary	Responsibility	Non-goal / risk	Validation metric
UI projection	Renders state and actions for humans.	Must not own canonical identity or proof.	Projection consistency; stale projection detection.
Filesystem API	Canonical access discipline for persisted user-owned data.	Must not bypass source/cache taxonomy.	direct_storage_bypass_count; readback_success_rate.
Parser-service	Executes parsing, query, operator, diagnostics and orchestration APIs.	Must not become hidden source of truth.	route coverage; health visibility; circuit breaker events.
Directus/PostgreSQL	Cache/index/auth/admin where applicable.	Must remain rebuildable and non-canonical for knowledge.	cache_rebuild_time; source/cache mismatch count.
LLM provider	Performs bounded transformations.	Must not own memory, proof or enterprise truth.	provider exposure coverage; output schema pass rate.
Observatory	Exposes runtime and proof/error state.	Must not manufacture proof from UI events.	error envelope coverage; proof state visibility.
CI/guards	Detect drift before merge or release.	Must classify blocking vs advisory to avoid noise.	guard pass rate; false-positive/false-negative review.

CTO / investor / governance / government review standard

Review audience	Adequacy standard	Evidence in document
CTO / engineering	Can reconstruct runtime topology, dataflow, service boundaries, tests, deployment risks and hardening path.	Runtime topology table, dataflow contract, guard catalogue, security readiness.
Investor / strategic	Can see that defensibility is not a model claim but an integrated stack of custom languages, contracts and operational controls.	Moat-code, moat-custom, operational moat, roadmap and benchmark bundles.
Governance / risk	Can distinguish proof, projection, export, review, authority and generated surfaces.	Forbidden equivalences, maturity tags, Error Truth, Authority Matrix, GovernanceRun.
Government / audit-facing review	Can identify source-of-truth, retention/exposure boundary, human decision points, incident/error lifecycle and recovery evidence.	Source/cache taxonomy, provider exposure, audit logs, proof bundles, backup/restore metrics.

Infrastructure adequacy rule

A-OSP should be considered infrastructure-described only when the document exposes: browser runtime, frontend shell, app registry, state layer, data access layer, parser-service boundary, source/cache split, realtime diagnostics, AI provider adapters, CI/CD control plane, deployment/security posture and recovery path. This chapter closes that gap and is cross-referenced by Annex J, Annex M and Annex O.

7. Text-First Virtual Epistemic Filesystem

The durable epistemic state is text-backed, append-only and local. Databases, UIs, providers and exports are derived or processing surfaces. This is not nostalgia for files; it is an engineering choice for portability, auditability, recovery, diffability and provider independence.

Anti-lock-in dependency contract
Only .aosp.txt owns durable knowledge. Directus/PostgreSQL is cache/index. LLM providers are stateless processors. React UI is projection/workspace. Exports are downstream representations. Generated docs are derived views. Proof belongs to readback, receipt, witness and proof bundle semantics.

Status taxonomy

```
Source:
 .aosp.txt
Cache:
 Directus / PostgreSQL
View:
 UI, Finder, Graph, Artifact Workstation
Adapter:
 LLM provider, Directus bridge, export layer
Export:
 PDF, DOCX, Markdown, JSON
Proof:
 receipt, readback, witness, proof bundle
```

[ILLUSTRATIVE] Canonical stripe example

```
atom_001|type:risk|label:Data breach risk|source:D1|session:SES-001|timestamp:2026-05-24T10:30:00Z
enrichment_001|parent:atom_001|kb_ref:ISO27001-A.12|confidence:0.85|source:D2
link_001|from:atom_001|to:atom_003|type:causal|source:D3
```

8. Stripe Schema Registry and Dynamic Ontology

The Stripe Schema Registry is the operational ontology of A-OSP. It makes stripe prefixes discoverable, validatable, renderable and queryable across app lenses. This is where the phrase semantic platform becomes concrete: prefix -> schema -> fields -> references -> validation -> projection.

Schema family	Examples	Why it matters
Core workflow	session_, answer_, question_, atom_, digest_	Basic epistemic capture and transformation.
Relation and synthesis	link_, lineage_, kernel_, gate_	Reticular dependency and compact synthesis.
Governance/proof	audit_, artifact_, consent_, completion_	Reviewable boundaries and downstream artifacts.
Moat-code	fae_, monl_, provider_, llm_call_	Orchestration, operator and provider surfaces.
Telemetry/runtime	telemetry_, budget_, pipeline_	Operational state and measurable economics.

Dynamic schema discovery
Unknown stripe types can be inspected and minimally typed instead of crashing the workspace. But dynamic discovery must be governed: dynamic schema -> quarantine -> review -> promotion to core schema -> generated control-plane update. Otherwise it becomes ontology drift.

KPI	Purpose
registered_schema_count	Measures typed ontology surface.
unknown_prefix_rate	Detects uncontrolled vocabulary drift.
schema_validation_pass_rate	Measures parsing/validation health.
schema_to_surface_coverage	Shows whether schemas are visible in app lenses.
directus_mapped_schema_count	Separates source schemas from cache/index mapping.

9. Multi-Abstraction Filesystem

The filesystem is not only storage. It can express evidence, absence of evidence, enrichments, relations, evaluations, compact kernels, artifacts, receipts, errors and governance states in one inspectable substrate.

Concept	A-OSP meaning	Non-goal
Filesystem	Durable local substrate of typed text-backed objects.	Not arbitrary folders of prose.
Ontology	Typed operational vocabulary and relations.	Not a rigid universal ontology.
Graph	Navigable relation layer over objects.	Not necessarily a graph database.
Semantic layer	Meaning carried by prefix, schema, relation, source and proof status.	Not just embeddings.

Knowledge base	Reviewed/promoted canonical objects.	Not automatic truth.

[ILLUSTRATIVE] Multi-abstraction object chain

```
answer_001
  "Payments above EUR 10k require CFO approval."

atom_001
  type: control_exists
  label: CFO approval threshold exists
  source: answer_001

atom_002
  type: evidence_gap
  label: No formal exception-handling procedure found
  source: answer_001

enrichment_001
  parent: atom_001
  kb_ref: internal_control_policy.payment_approval

link_001
  from: atom_002
  to: atom_001
  type: weakens_control_reliability

score_001
  target: control.payment_approval
  maturity: partial

kernel_001
  stance: partially governed control with evidence gap on exceptions
  depends_on: [atom_001, atom_002, enrichment_001, link_001, score_001]
```

10. Local, Team and Enterprise Source-of-Truth

A-OSP should not treat every local object as enterprise truth. It should support a promotion model: local/session objects become team or enterprise canonical objects only after review, diff, conflict handling, receipt and governance decision.

Layer	Objects	Governance rule
User/session SSOT	local answers, atoms, evidence gaps, local kernels, draft artifacts	Useful for work-in-progress; not enterprise truth.
Team/workflow SSOT	reviewed objects, workflow kernels, accepted control facts, unresolved conflicts	Owned by workflow/reviewer context.
Enterprise canonical SSOT	approved control library, canonical policies, reviewed knowledge base, evidence map	Requires promotion decision, diff and rollback path.

Local-to-enterprise promotion path

```
User/session local SSOT
  -> team review
  -> diff / conflict handling
  -> promotion receipt
  -> enterprise canonical SSOT
  -> downstream artifacts / audits / benchmarks
```

PART III - QUERY, ORCHESTRATION AND PROOF

EQL makes state queryable; FAE makes model calls bounded; MONL makes parsed stripes transformable; proof contracts make transitions reviewable.

11. EQL, Scoped Retrieval and Certified Query Surfaces

Before a model is called, A-OSP needs scoped retrieval: which persisted objects are relevant to this operation? EQL is the epistemic query language for this role. It should be understood not as SQL-with-extras, but as an A-OSP DSL for append-only, session-scoped, stripe-shaped datasets with epistemic guarantees.

Level	Role	Proof relevance
EQL-Core	Read-only object retrieval over real prefixes.	Baseline query surface.
EQL-Proof	Canonical session scope, proof envelopes, virtual proof views.	Required for proof-sensitive retrieval.
EQL-Analytic	Aggregation and lineage tracing.	Useful for audit, metrics and reticular analysis.

EQL proof query example

```
[CONTRACT / HARDENING]
PROOF SELECT * FROM atom_ IN SESSION 'SES-001' WHERE type = 'risk'


Expected proof metadata:
 query_scope
 query_hash
 result_hash
 proof_eligible
 EQLQueryReceipt
```

Certified query surfaces
Every surface that exposes EQL should declare exactly what it supports. Shell, Finder, EQLSearchBar, FAE, iKantAssistant and parserService should not suggest features the parser or surface cannot execute with the same guarantees.

Current-state boundary. EQL L0-L2 foundations are landed: capability matrix, certified examples, parser, virtual views and tokenizer. Proof-grade EQL is not yet complete until EQLRunResult, EQLQueryReceipt, canonical hash, unified route and proof CI are implemented. See Appendix P.7-P.10.

KPI	Purpose
eql_certified_examples_count	Measures certified DSL coverage.
eql_surface_coverage	Shows which app surfaces expose EQL safely.
eql_unsupported_rejection_count	Measures typed rejection rather than silent fallback.
eql_query_receipt_coverage	Measures proof-grade query adoption.
trace_query_success_rate	Measures lineage query health.

12. FAE and Bounded Model Orchestration

FAE profiles are A-OSP phase-level orchestration objects. They specify Focus, Attention and Execution: which objects are selected, which thresholds or constraints apply, and which provider/model/schema/fallback policy runs the transformation.

FAE dimension	Meaning	Example
Focus	Which objects are selected, usually through EQL.	EQL SELECT answer_ WHERE session=$SESSION
Attention	Parameters and constraints for the phase.	min_conf=0.65, max_atoms=10
Execution	Model/provider/schema/fallback behavior.	model, temp, max_tokens, fallback=error

FAE canonical stripe profile

```
[CONTRACT / HARDENING]
fae_[phase:D1 :
   name:atomic_extraction :
```

```
focus:EQL SELECT answer_ WHERE session=$SESSION :
attention:min_conf=0.65/max_atoms=10 :
execution:model=gpt-4o,temp=0.3,max_tokens=2000,fallback=error]
```

Economics hypothesis
Cost reduction is not claimed as proven. A-OSP creates the measurement substrate to test whether evidence atomized once, compact kernels, scoped references and provider substitution can reduce repeated model-call cost and human rework in selected workflows.

13. MONL Operators and Conditional Transaction Membrane

MONL should not be over-emphasized as a user-facing language. Its value is as an operator layer over parsed stripes: filtering, mapping, grouping, extracting, graph-building, relation-finding, merging and diffing A-OSP source objects. It becomes proof-critical only if contracted as the transformation membrane for append/query/readback/diff semantics.

MONL operator examples

```
[CONTRACT / HARDENING]
filter(ast, stripe => stripe.prefix === 'atom_')
groupBy(ast, stripe => stripe.session)
diff(previousAst, currentAst)
buildGraph(ast)
findRelations(ast, objectId)
```

KPI	Purpose
monl_operator_coverage	Breadth of operator library.
monl_diff_success_rate	Reliability of diff/promotion workflows.
monl_graph_build_success_rate	Graph/reticulum construction health.
monl_proof_critical_operator_count	How much of MONL is actually proof-sensitive.
monl_adapter_bypass_count	Detects unsafe bypass of operator membrane.

14. Operational Proof Model

Proof is a chain, not a badge. A proof-sensitive transition is not complete because the UI advanced. It becomes proof-grade only when the object was persisted, read back, compared, represented by receipt metadata and exposed as proof state.

[CONTRACT / HARDENING] Proof transition

```
proof-sensitive intake
 -> persist object
 -> read back object
 -> compare persisted/read object
 -> emit receipt
 -> expose proof status
 -> downstream consumption
```

Proof object	Role	Maturity
D1Receipt	Receipt for proof-sensitive intake.	HARDENING
D1ProofMode	Stricter mode blocking false-green demo/fallback path.	HARDENING
EQLQueryReceipt	Scope/hash/resultHash for proof query.	TARGET
PhaseReceipt	D2-D5 phase proof object.	TARGET
PipelineReceipt	Composed pipeline lineage receipt.	TARGET
Proof Spine	Composition layer across receipts, witness and governance.	TARGET

Why Proof Spine is needed
A single receipt validates one operation. A real artifact depends on many operations: intake, retrieval, enrichment, transformation, artifact assembly and review. The Proof Spine is the target composition layer connecting these proof objects into one reviewable chain.

Current-state boundary. D1Receipt and D1 proof-mode contracts are landed, and DataIntake UI is fail-closed against false-green states. However, the backend D1 proof producer, persisted d1_receipt_ stripe and read-after-write barrier are not yet complete. D1 proof PASS must not be externally claimed until the #3462 proof seam and #3488 C1 checkpoint are closed. See Appendix P.3-P.6.

15. Artifact Boundaries and GovernanceRun

Generated document does not equal proof. A reviewable artifact must expose sources, lineage, receipts or witness, unsupported claims, missing evidence and review status. ArtifactReceipt and ArtifactWitness are related but not identical.

Object	Meaning	Maturity
ArtifactReceipt	Proof-boundary receipt for sealing/exporting an artifact.	HARDENING
ArtifactWitness	Dependency map explaining sources, unsupported claims and missing evidence.	TARGET
GovernanceRun	Review context consuming proof objects and recording decision/follow-up.	TARGET

GovernanceRun example

```
[TARGET]
{
  "governance_run_id": "gov.payment_controls.001",
  "artifact_id": "artifact.payment_controls.001",
  "consumed_proof_objects": [
    "d1.receipt.atom_001",
    "pipeline.receipt.payment_controls.001",
    "artifact.receipt.payment_controls.001"
  ],
  "unsupported_claims": ["Formal exception-handling procedure is approved"],
  "decision": "accepted_with_follow_up",
  "follow_up_actions": ["upload formal procedure or mark control gap"],
  "reviewer_role": "compliance_reviewer"
}
```

PART IV - HUMAN OPERATING LAYER AND ERROR TRUTH

The system must be operable and diagnosable by humans; proof contracts are useless if their status is invisible.

16. OS-like Workstation and App Lenses

OS-like does not mean native OS or desktop clone. It means coordinated operating surfaces over one epistemic substrate: capture, search, inspect, relate, assemble, govern, diagnose and configure.

App lens	User function	Primitive underneath
DataIntake	capture evidence and answers	stripe write, D1, receipt
Finder	retrieve objects	EQL, schema registry
Inspector	inspect source/proof/lineage	schema, lineage, receipts
Graph	relate objects	links, MONL/EQL graph
Artifact Workstation	assemble reviewable outputs	kernel, witness, receipt
Governance	review/approve/follow-up	GovernanceRun, Proof Spine
Observatory	diagnose system/proof failure	Error Truth, telemetry
Settings	configure provider/capability/runtime	provider, consent, capability state
System Trail	reconstruct chronology	events, receipts, errors

App Support Tier Matrix
Tier-0 apps form the core operating path. Tier-1 apps are supported user-facing surfaces. Tier-2 apps support diagnostics, artifacts and observability. Tier-3 apps are experimental and must be visibly labeled as such. This prevents experimental surfaces from being mistaken for canonical proof paths.

17. Observatory, Error Truth and AOSPErrorEnvelope

Observatory is the diagnostic plane. Error Truth is the forensic epistemic contract behind it. A generic error toast is not enough: the system must identify what happened, where, what is impacted, whether it is fatal/degraded/missing/blocked/recoverable, what proof/log/context exists, what action is available and whether it is the same problem already seen elsewhere.

AOSPErrorEnvelope shape

```
[CONTRACT / TARGET]
interface AOSPErrorEnvelope {
  envelopeId: string
  rootCauseId: string
  appId: string
  origin: string
  transport: 'http' | 'websocket' | 'filesystem' | 'worker' | 'unknown'
  code: string
  severity: 'info' | 'warn' | 'error' | 'critical' | 'fatal'
  proofRefs?: string[]
  state: 'open' | 'acknowledged' | 'recovered' | 'suppressed' | 'expired' | 'closed'
}
```

Local truth / global truth
A local app needs an immediate actionable projection. Governance needs a durable canonical envelope or proof object. The same distinction applies to errors, evidence, artifacts and enterprise promotion.

KPI	Purpose
error_envelope_coverage	Share of errors converted into canonical envelopes.
root_cause_dedup_rate	Measures deduplication quality.
opaque_http_error_count	Detects unclassified backend failures.
websocket_error_attribution_rate	Prevents generic connection failure.
recovery_action_coverage	Measures actionability.

18. UX Honesty, Projection Model and Capability States

UX Honesty depends on Observatory. A UI cannot be honest if proof state, fallback state, backend health, capability state and transport state are invisible. UI cards, dashboards and exports may present proof or error state, but they do not own canonical identity.

Projection model

canonical object owns identity
UI owns projection
export owns representation
proof owns contract

Capability state	Meaning
ready	Capability is available in current scope.
missing_config	Required config is absent, such as provider key.
unavailable	Capability cannot be reached.
degraded	Capability works with reduced guarantees.
rate_limited	Provider or service constrained by quota.
permission_denied	User/browser/role denied required permission.
unknown	State cannot be classified yet.

UX honesty rules
green requires proof state
fallback must be visible
placeholder surfaces must be labeled or removed
export without witness is non-proof
capability unavailable is not the same as proof failure
projection must not become source of truth

D1/EQL proof UX boundary. DataIntake currently preserves honesty by failing closed: questionnaire completion does not become D1 proof completion, and D2 continuation is blocked unless proof state is green. EQL proof UX remains incomplete: cross-surface status cards, scope badges, proof badges, zero-result explanations and mode chips are still target surfaces. See Appendix P.8

PART V - REPOSITORY AS CONTROL PLANE

A-OSP must govern its own AI-assisted development process to be credible as a governance platform.

19. Repository Evidence, Authority Matrix and Generated Control Plane

A-OSP treats repository governance as part of the product architecture. The repository is not merely where the product is stored; it is also where authority, drift, proof boundaries, generated documentation and implementation debt are made visible.

Metric	Value / target	Interpretation
Repository files	4,686	Real project scale.
Code files analyzed	2,598	Implementation footprint.
Graph nodes	7,557	Dependency/semantic surface.
Graph edges	16,304	Coupling and audit need.
Imports	11,583	Integration complexity.
Functions/classes	4,848	Behavior surface.
Unresolved imports	137	Hardening debt.
Potential dead / entrypoint-opaque	93	Cleanup/legacy risk.
Audit findings global	305 IDs / 32 actions	Known risk universe and deduplicated action plan.

Authority Matrix
Authority Matrix is A-OSP repository-level epistemic map. It says which files are source-of-truth, which are generated, which are advisory and which are archaeological. Generated docs must remain read-only, regenerable and checkable.

Generated control plane
Authority Matrix, app inventory, support tiers, EQL capability matrix, certified examples and Knowledge Center documents are not decorative prose. They are derived views over source artifacts and should be regenerated, checked and treated as read-only.

20. PR, Issue, Guard and Bot-Control Lifecycle

In mature A-OSP development, an issue is not merely a task and a PR is not merely a patch. Together they form a bounded epistemic transaction over repository state.

Guarded development lifecycle

```
Issue draft
  -> task scope
  -> acceptance criteria
  -> references / roadmap / decisions
PR draft
  -> Bussola
  -> local/global trade-off
  -> plain-language explanation
  -> issue reference
  -> D-030 compliance
  -> test requirements
Guard execution
  -> build/test
  -> E2E
  -> Knowledge Center sync
  -> dataflow / D-030
  -> security / freeze
  -> topology / UI / a11y checks
AI reports
  -> bot summary
  -> ADV classification
  -> coordination gate
  -> topology validation


Human review
  -> missing task detection
  -> retry prompts
  -> non-overlap guard
  -> merge / revert / follow-up issue
```

Control	Purpose
Bussola	Past / present / future / vision narrative control.
Agent rules	Prevent scope drift and stale authority usage.
PR template	Force issue link, tradeoff, validation and docs/test mapping.
Bot reports	Human-facing control surface, not autonomous authority.
Entropy protocol	Detect drift, duplicates, fake completeness and unmanaged complexity.
Convergence protocol	Turn findings into deduplicated actions and roadmap gates.

21. Test-and-Guard Architecture

A-OSP trust posture is not a single green check. It is a layered mesh: unit tests, integration tests, E2E tests, golden-path tests, accessibility checks, dataflow guards, topology validation, generated-doc checks, storage SSOT linting, governance scripts and human review.

Test-and-guard mesh

```
unit tests
  + integration tests
  + E2E tests
  + golden-path tests
  + accessibility tests
  + dataflow guards
  + generated-doc checks
  + topology validation
  + proof-negative tests
  + human review
```

Metric	Purpose
unit_test_count	Implementation correctness surface.
integration_test_count	Cross-module behavior.
e2e_test_count	User journey behavior.
golden_path_count	Proof-critical reference paths.
guard_script_count	Governance coverage.
false_green_regression_count	Product honesty.
generated_doc_sync_pass_rate	Control-plane integrity.

22. Enterprise Deployment, Security and Provider Exposure

Enterprise readiness depends on identity/session boundaries, role tests, backup/restore, health monitoring, runbooks, tenant boundaries, provider exposure controls and browser capability scope.

Provider exposure boundary
For each model call, A-OSP should record allowed_refs, prompt/context payload class, data sensitivity class, provider, retention assumption, output validation status and human review requirement.

Security and operations controls should be validated as explicit metrics rather than assumed from deployment posture.

Security / operations control	Metric
Role boundaries	role_permission_test_pass_rate
Tenant isolation	tenant_isolation_test_pass_rate
Browser permission scope	permission_revocation_success_rate
Backup/restore	backup_restore_success_rate
Cache rebuild	cache_rebuild_time
Provider exposure	model_call_data_classification_coverage
Audit logs	audit_log_completeness

PART VI - VALIDATION, ROADMAP AND STRATEGIC VALUE

The project must remain measurable, falsifiable and clear about what is AS-IS, HARDENING, TARGET or HYPOTHESIS.

23. RLA/ECNN as Lightweight Theoretical Grammar

A-OSP uses RLA/ECNN as an engineering grammar, not as metaphysics. RLA provides levels and transmissions; CRC provides compactness, epistemic closure and computability; ECNN provides explicit unknown/contradiction channels; ECU/UCE provides bounded epistemic units that produce artifacts under deterministic constraints. Public theoretical reference: https://github.com/Luke883i/RLA-ECNN

RLA/ECNN concept	A-OSP translation
levels	D1-D5, app lenses, proof states
transmissions	append, query, transform, witness, promote
non-injective collapse	summaries, kernels, exports, UI views
unknown	evidence_gap / does not know
contradiction	conflict object / uncertainty signal
ECU/UCE	bounded model call under deterministic constraints
Popper-style challenge	negative tests and false-green tests

Why non-injective collapse matters
Summaries, kernels, exports and UI views are not lossless truth. They compress, omit or reweight information. This is why A-OSP needs source references, missing-evidence objects, contradiction handling, proof boundaries and negative tests.

24. Compliance as First Wedge, Platform Beyond Compliance

Compliance is the first wedge because it stress-tests evidence, absence of evidence, accountability, formal artifacts and human review. A-OSP is not compliance certification; it is infrastructure for proof-sensitive workflows that may support compliance, audit and risk work.

Use case	Why A-OSP may fit
Supplier payment control	Evidence, thresholds, gaps, review and artifact witness.
Supplier due diligence	Multi-source evidence, browser capture, risk flags.
Incident review	Logs, timeline, root-cause hypotheses, corrective action.
Policy impact analysis	Changed policy, impacted controls, unsupported claims.
Technical audit	Repository evidence, test/guard outputs, proof bundles.
Research synthesis	Source references, missing evidence, bounded synthesis.
AI-assisted project governance	Issue/PR/guard lifecycle and authority matrix.

Normative kernel pattern
For governance-heavy workflows, A-OSP can distinguish world state (what evidence says), self state (what the organization has declared or done), and normative kernel (which rules, policies or obligations apply). This is inspired by the RLA-ECNN / iKant research line and remains a target grammar, not a legal authority.

25. Roadmap: Proof Convergence and Moat-Custom Maturation

Track	Objective	Expected evidence
C0	Corpus authority, anti-drift, proof scaffolding	Authority Matrix, generated control plane, no stale authority.
C1	D1 + EQL proof seam	D1Receipt, EQLQueryReceipt, proofSessionContext.
C2	D2-D5 lineage	PhaseReceipt and PipelineReceipt.
C3	ArtifactWitness + Governance composition	ArtifactBundle and GovernanceRun.
C4	Error Truth + UI proof honesty	AOSPErrorEnvelope, capability/projection state.
C5	MONL conditional proof membrane	Only if transformation semantics become proof-critical.
GOV	Entropy, Convergence and Agent Governance	PR/issue/guard lifecycle, generated control checks.
SCHEMA	Stripe Registry hardening	schema validation, dynamic schema quarantine.

APP	App Support Tier enforcement	tier-0 E2E, tier labels, surface parity.

D1/EQL Proof-Grade Reality Check

The first proof-convergence milestone is intentionally narrow: D1 intake and EQL retrieval must become proof-grade before A-OSP can credibly claim downstream ArtifactWitness, Proof Spine or GovernanceRun readiness. **This section records the current engineering reality. It is not a product-completion claim. It is a maturity boundary.**

As of the D1/EQL engineering attestation dated 2026-05-26, at reference commit **aa68898cd1b882549e8b354aad894dc86629566e**, D1/EQL proof-grade is not yet reached. The repository already contains substantial landed foundations: shared proof contracts, D1 receipt schemas, D1 proof-mode primitives, DataIntake UI honesty logic, EQL capability matrix, certified examples, strict parser, virtual views and tokenizer. However, the closed end-to-end proof seam is still incomplete.

Current maturity distinction:

```
D1/EQL foundations:      AS-IS / LANDED
D1 UI honesty:           AS-IS / RUNTIME-WIRED / fail-closed
D1 proof producer:       MISSING / HARDENING target
D1 persisted receipt:    MISSING / HARDENING target
D1 readback barrier:     MISSING / HARDENING target
EQL proof run envelope:  MISSING / HARDENING target
EQL query receipt:       MISSING / HARDENING target
EQL proof UX:            MISSING / TARGET, with demo-readiness relevance
C1 convergence artifacts: MISSING / final M1 gate
```

The critical distinction is:

```
questionnaire complete != D1 proof complete
generic append success != proof PASS
validated atom != receipt-backed proof
raw EQL success != proof query
zero rows != failure unless expectedMinRows requires rows
UI green != proof unless backed by receipt and readback
```

D1 proof should mean:

```
D1Receipt PASS
persisted d1_receipt_
readback hash match
explicit session scope
no generic append can create proof PASS
D2 handoff blocked without D1Receipt + readback
Finder/Shell receive explicit proofSessionContext
```

EQL proof should mean:

```
EQLRunResult envelope
EQLQueryReceipt
canonical query
astHash / queryHash / resultHash
explicit scope
proof mode rejects implicit/global proof queries
raw query success is not treated as proof
```

UX proof should mean:

```
DataIntake can show green only from proof PASS
fallback/demo/local modes cannot become proof
CompletionModal blocks D2 without proof state
Finder/Shell expose scoped session proof context
EQL surfaces expose parse/capability/scope/result/proof status
VALID_ZERO is distinguished from FAIL_EMPTY
```

The attached D1/EQL engineering attestation maps the current state across UX, UI, contracts, SDK, backend, store, readback, EQL evocation and EQL receipt. Its conclusion is deliberately strict:

```
D1/EQL proof-grade is not complete today.
The foundation is substantial.
The proof seam is precise.
The planned issue chain is adequate.
The final claim must be gated by C1 convergence artifacts.
```

The issue-chain coverage is complete: 14/14 identified proof-grade gaps are owned by open issues. This makes the roadmap credible, but it does not convert planned work into completed proof. External proof-grade claims should wait for the C1 convergence gate.

For the compressed engineering inventory, issue-chain assessment, adequacy review and closure DAG, see Appendix P - D1/EQL Proof-Grade Engineering Attestation.

26. Metrics, Bundles and Benchmark Plan

Bundle	What it measures
RepositoryBundle	repo scale, audit findings, authority state, CI/guard state
SchemaBundle	registered schemas, unknown prefixes, validation pass rate
D1ProofBundle	intake persistence, readback, D1Receipt coverage
EQLQueryBundle	scope/hash/resultHash and proof eligibility
PipelineBundle	D2-D5 phase receipts and lineage
ArtifactBundle	artifact witness, unsupported claims, missing evidence
GovernanceBundle	Proof Spine + GovernanceRun + human review
ErrorTruthBundle	AOSPErrorEnvelope coverage and root-cause dedup
AppTierBundle	tier support, surface labeling, tier-0 E2E
EconomicBundle	tokens, calls, review time, correction rate, provider delta
ProviderExposureBundle	payload class, provider, retention, sensitivity and validation

Proof/economic bundle shape

```
[TARGET]
{
  "proof_bundle_id": "proof.session.SES-001",
  "source_objects_count": 42,
  "receipt_count": 11,
  "readback_count": 11,
  "lineage_edges": 38,
  "unsupported_claims_count": 2,
  "missing_evidence_count": 3,
  "input_tokens_total": 18420,
  "output_tokens_total": 2670,
  "reused_context_objects": 31,
  "new_llm_calls": 3,
  "status": "review_required"
}
```

27. Risks and Compensating Controls

Risk	Control
Founder/team dependency	OS-like workstation, Observatory, generated control plane, runbooks.
Contract-only ambiguity	Claim discipline and maturity tags.
False-green UI	D1ProofMode, Error Truth, UX honesty tests.
Dynamic schema drift	quarantine/review/promotion of dynamic schemas.
Authority matrix staleness	generated-doc sync and authority check.
Projection identity confusion	canonical object vs UI projection rule.
Capability state ambiguity	capability grammar and AOSPErrorEnvelope.
Guard noise	severity classification and actionability metrics.
Economic hypothesis unproven	benchmark plan and EconomicBundle.
Browser capability overreach	explicit user permission, scope, audit and revocation.
D1/EQL proof-grade overclaim	Appendix P claim boundary, #3462 D1 proof seam, #3425-#3430 EQL proof chain, #3488 C1 checkpoint.
EQL proof invisible to users	Minimal EQL proof UX surface before external demo; full EQLStatusCard / ScopeBadge / ProofBadge in #3428.

What would falsify the thesis?
A-OSP would fail its own thesis if users cannot diagnose failed proof states; the filesystem cannot be reliably parsed/rebuilt; artifacts cannot trace claims to source objects; bounded calls do not reduce rework or review effort in measured workflows; local-to-enterprise promotion creates unmanageable conflicts; or guard systems produce noise without actionable repair.

28. Strategic Validation Thesis and Conclusion

A-OSP should be judged by whether it can make AI-assisted work reconstructable, diagnosable, portable, bounded, auditable and reviewable across real enterprise workflows. The strategic validation thesis is that a browser-native, text-first epistemic operating environment can reduce proof ambiguity, provider lock-in and repeated context reconstruction in selected workflows. The next validation step is to measure this through proof-grade D1/EQL, artifact boundaries, Error Truth, Observatory metrics and pilot workflow benchmarks.

Category	Strength	Gap	A-OSP wedge
Chatbot	generation	weak proof/lineage	proof-and-knowledge substrate
Agent framework	automation	weak durable local SSOT	text-first filesystem + guards
GRC tool	workflow governance	not AI-native proof	evidence gaps + receipts
Vector DB	retrieval	weak governance semantics	multi-abstraction object model
Document generator	output	weak witness/review boundary	ArtifactReceipt/Witness
Data room	evidence storage	weak bounded model orchestration	scoped calls + proof bundles

APPENDIX A – O

Operational catalogues, engineering attestations and review standards for engineering, CTO and governance review.

Appendix A - Moat-Custom Catalogue

Moat-custom asset	Function	Maturity	Failure prevented	Primary KPI
Stripe Schema Registry	Operational ontology for prefixes/schemas.	AS-IS/HARDENING	Unknown prefix becomes untyped prose.	registered_schema_count
Dynamic Schema Discovery	Infer minimal schemas from parsed stripes.	AS-IS/RISK-CONTROL	Ontology drift or crash on new object.	unknown_prefix_rate
EQL	Epistemic query DSL.	CONTRACT/HARDENING	Unsupported query treated as proof-ready.	eql_query_receipt_coverage
FAE profiles	Focus-Attention-Execution orchestration.	HARDENING	Generic prompt becomes unbounded model call.	fae_output_schema_pass_rate
MONL operators	Parsed-stripe transform/diff/graph layer.	CONDITIONAL/TARGET	Ad hoc transformation bypasses proof membrane.	monl_diff_success_rate
Error Truth Contract	Forensic error model.	CONTRACT/P0	Opaque unrelated UI errors.	error_envelope_coverage
AOSPErrorEnvelope	Canonical error object.	TARGET	No root-cause identity or lifecycle.	root_cause_dedup_rate
Authority Matrix	Repository authority map.	AS-IS	Stale doc treated as source truth.	authority_matrix_check_pass_rate
App Support Tier Matrix	Product maturity map.	AS-IS	Experimental app appears canonical.	tier0_e2e_coverage
Generated Control Plane	Derived docs/checks as control surfaces.	HARDENING	Docs drift from source artifacts.	generated_doc_sync_pass_rate
Projection Model	Canonical identity vs UI/export representation.	CONTRACT/TARGET	UI card becomes source truth.	projection_consistency_rate
Capability State Model	Runtime capability grammar.	CONTRACT/TARGET	Fake green or fake failure.	capability_state_coverage
Meta-controller Navigator	Reticular governance over explicit objects.	TARGET	Dashboard claims global truth.	horizon_violation_count

Appendix B - Concept Injection Matrix

Concept	First definition	Engineering section	Snippet/KPI section	Cross-reference
Ontological Semantic Platform	Ch.3	Ch.8-8	Schema/KPI tables	EQL, proof, registry
Browser-native substrate	Ch.5	Ch.16/21	Browser stack	App lenses, capability
Text-first SOT	Ch.7	Ch.14/25	Stripe/recovery snippets	Anti-lock-in, proof
EQL	Ch.11	Ch.11/11	Proof query	FAE, proof, economics
Error Truth	Ch.17	Ch.18/26	AOSPErrorEnvelope	UX honesty, Observatory
Authority Matrix	Ch.19	Ch.20/20	Generated control plane	Guard lifecycle
Local/enterprise SSOT	Ch.10	Ch.15/23	Promotion path	GovernanceRun

Appendix C - Epistemic Object Lifecycle

```
draft
  -> persisted
  -> readback-confirmed
  -> receipt-backed
  -> review-required
  -> accepted
  -> promoted
  -> superseded
  -> rolled-back
```

Appendix D - Human Roles Matrix

Role	Responsibilities
Operator	Captures evidence and runs workflows.
Reviewer	Approves, rejects or requests evidence.
Governance owner	Defines policy/control boundaries.
Technical operator	Monitors deployment, health, backup and roles.
Auditor	Inspects proof bundles and review history.

Appedinx E - Operational Moat Map

Scripts, workflows, guards, policies and generated reports that keep the architecture from drifting.

Moat asset	Layer	Failure prevented	Primary KPI
Data Foundation Guard	CI/CD	Data foundation drift across schema, mapping, hooks, app data usage, stripe sync and source policy.	guard_pass_rate
Schema Registry Guard	CI/CD / schema	Unregistered schema drift or unknown prefixes silently entering runtime.	schema_validation_pass_rate
Entity Mapper Guard	CI/CD / data	Missing mapping between stripe schemas, entities and render/query surfaces.	entity_mapping_coverage
Store & Hooks Guard	CI/CD / frontend	Broken query/mutation behavior in the data access layer.	store_hook_test_pass_rate
App Discovery Guard	CI/CD / app layer	Untracked app data usage and hidden app surfaces.	uncovered_app_count
Stripe Sync Guard	CI/CD / storage	File/cache desynchronization and missing stripe emission.	stripe_sync_pass_rate
Dataflow Contract Guard	CI/CD / architecture	D-030 filesystem-first violations and undocumented app consumes/produces.	dataflow_contract_violation_count
Typed Wiring Guard	CI/CD / typing	Type boundary gaps between schemas, UI, hooks and dataflow.	typed_wiring_violation_count
Read Source Policy Guard	CI/CD / routing	Wrong canonical/session source selection for prefixes.	read_source_policy_coverage
AI Dataflow Compliance	CI/CD / semantic review	Semantically wrong dataflow changes despite passing build/tests.	ai_dataflow_warning_count
D1 Golden Gate	E2E / proof	False D1 proof confidence and demo fallback masking.	d1_golden_pass_rate
E2E Flake Budget	E2E / determinism	Nondeterministic proof paths and unreliable critical journeys.	critical_journey_flake_count
Invariant Registry	Governance / architecture	Duplicated, invisible or unenforced architectural rules.	invariant_enforcement_coverage
AI Dataflow Guide	Agent governance	Agent uses stale or wrong data access pattern.	mandatory_reading_compliance
iKant ICP Gates	Agent governance	Unreviewed AI-assisted drift and missing process evidence.	icp_gate_failure_count
Conversational Guardrails	Agent governance	Quickfix, bypass, workaround and scope creep patterns.	guardrail_trigger_count
Transclusion Engine	Doc governance	Manual documentation synchronization drift.	transclusion_drift_count
Deep Clean Docs	Doc governance	Stale, duplicate, placeholder or dead-link documentation debt.	doc_anomaly_count
Strategic Coherence Check	Governance	Changes outside active task, phase, roadmap or decision boundary.	unauthorized_change_count
Classification Registry	Repository ontology	Dormant, dead or archaeological surfaces becoming invisible.	classification_coverage
Topology Validation System	Topology / architecture	Route/call/E2E mismatch and hidden topology drift.	topology_drift_count
Security Scan Pipeline	Security	Dependency, container, filesystem or configuration blind spots.	high_critical_vulnerability_count

Design rule
Every operational moat asset should have an owner surface, trigger condition, blocking/advisory status, failure mode, report artifact, KPI and roadmap or decision reference.

Appendix F - CI/CD Guard Catalogue

CI/CD is treated as a control plane: each guard protects a specific architecture claim.

Guard family	Guard / job	Protected claim
Data foundation	schema-registry-guard	Stripe schemas are registered, valid and discoverable.
Data foundation	entity-mapper-guard	Entities can round-trip through mapping and are not orphaned.
Data foundation	store-hooks-guard	DataStore and hooks preserve query/mutation contracts.
Data foundation	app-discovery-guard	App data usage cannot remain invisible.
Data foundation	stripe-sync-guard	Stripe emission and file/cache synchronization remain testable.
Data foundation	dataflow-contract-guard	D-030 dataflow contract blocks architectural violations.
Data foundation	typed-wiring-guard	Typed boundaries between schema, data and UI are audited.
Data foundation	read-source-policy-guard	Prefixes are routed through declared source policy.
Data foundation	ai-dataflow-compliance	AI-assisted semantic review checks app/data changes.
Data foundation	report / badge	Guard outcomes are visible to humans and PR reviewers.
E2E	critical-journeys	Tier-0/1 paths are deterministic enough for review.
E2E	d1-golden	D1 proof promise has a master gate.
E2E	apps	App lens behavior and cross-app flows are covered.
E2E	a11y	Accessibility regressions are surfaced.
E2E	navigation	Keyboard/navigation behavior remains testable.
E2E	ikant-modes-e2e	Governance/assistant surface transitions are covered.
Lint / static	storage-ssot	Session entities cannot bypass useFilesystem.
Lint / static	no-monl-adapter-session-prefix	Manual session stripe writes cannot bypass SSOT.
Lint / static	d1mode-ssot	D1 mode state cannot fork.
Lint / static	no-d1-fire-and-forget	D1 errors cannot be swallowed.
Lint / static	atom-types-ssot	Atom type vocabulary remains centralized.
Lint / static	kb-mapping-coverage	Knowledge base mapping coverage is checked.
Lint / static	stripe-prefix-coverage	Prefix registry coverage is checked.
Lint / static	no-test-only-globals	Test-only globals do not leak into production paths.
Lint / static	no-electron-imports	Browser-native architecture is preserved.
Documentation	validate-docs	Required docs, metadata, reachability and links remain coherent.
Documentation	validate-integration	ROADMAP, blueprint and development line stay synchronized.
Documentation	update-registry	Documentation registry refreshes from source.
Documentation	archive-obsolete	Obsolete docs are archived with traceability.
Documentation	deep-clean-docs	Stale, duplicate and placeholder docs are cleaned or quarantined.
Documentation	transclude	Source-to-sink markdown synchronization is checked.
Documentation	strategic-coherence	Changes align with active task, phase and roadmap.
Security	npm audit	Known npm vulnerabilities are detected.
Security	Trivy	Dependencies, containers, filesystem and misconfigurations are scanned.
Security	security checklist	Production blockers remain visible.

Catalogue rule

A guard should not be described as “quality” in general. It should be tied to a specific architecture claim and failure mode.

Appendix G - Test Matrix

Layered validation system; not all tests carry the same proof weight.

Test class	Tool / surface	Purpose	Proof weight
Unit tests	Vitest	Local behavior correctness for components, utilities and contracts.	Low-medium
Invariant tests	Vitest / custom scripts	Architectural rule enforcement and drift detection.	Medium-high
Integration tests	Playwright + backend	Service and backend-dependent flows.	Medium
Smoke E2E	Playwright	CI-safe product sanity on key pages and flows.	Medium
D1 golden	Playwright	D1 proof-sensitive master gate with deterministic behavior.	High
Critical journeys	Playwright	Tier-0/1 user journeys and proof-relevant paths.	High
Apps E2E	Playwright	App lens behavior and cross-app flows.	Medium
A11y	Playwright + axe	Accessibility constraints and regressions.	Medium
Navigation	Playwright	Keyboard/navigation behavior.	Medium
iKant modes	Playwright	Governance and assistant surface flows.	Medium
LLM smoke	Vitest	Provider integration sanity without treating output as proof.	Low-medium
Flake budget	custom script	Determinism control and flake tracking.	High
Seed baseline	custom script	Canonical input drift detection and baseline reproducibility.	High

Proof-critical test standard
Proof-sensitive tests should be deterministic, avoid static skips and arbitrary hard waits, use pinned seed inputs where relevant, fail when fallback masks proof failure, and produce human-readable reports.

Appendix H - Invariant Catalogue

Canonical rules that translate architectural principles into enforceable controls.

ID	Invariant	Enforcement surface	Failure prevented
INV-01	Text-first single source of truth	D-030 dataflow guard	DB/file inconsistency
INV-02	Append-only immutability	Parser/MONL/runtime audit	Audit trail corruption
INV-03	Filesystem-first data access	CI lint + dataflow contract	Direct Directus bypass
INV-04	Stripe format stability	Stripe registry tests	Unparseable existing stripes
INV-05	Pipeline execution order	Orchestrator/checkpoint validation	Incoherent D1-D5 graph
INV-06	DSL grammar preservation	Parser-service tests/versioning	Incompatible MONL/EQL change
INV-07	Session ownership	Ownership hook/backend enforcement	Unauthorized data access
INV-08	Deterministic prefix routing	useFilesystem routing + policy guard	Wrong canonical/session source
INV-09	Error propagation	queryWithStatus pattern	Silent empty-array fallback
INV-10	Audit trail completeness	useAudit/audit stripe validation	Compliance/action gap
INV-11	Persist-first atom lifecycle	Atom persistence hook	Data loss before validation
INV-12	Cross-app navigation contract	shell.openApp/useAppTransition + schema	Context loss or undocumented coupling
INV-13	CI gate integrity	Branch/workflow protection	Suppressed failing checks
INV-14	Layer dependency direction	Architecture review/topology	Circular fragile coupling
INV-15	No direct storage in apps/hooks	CI lint	Audit bypass/local storage drift

INV-16	Session-entity SSOT	Storage-SSOT lint/no adapter bypass	Invisible session writes
INV-17	Atom validation referential integrity	Pre-check + post-loop assertion	Orphan atom_validation_ stripe

Invariant rule
If a rule is important enough to appear in multiple documents, it should have one canonical invariant ID and all other documents should reference it.

Appendix I - AI Agent Governance Protocol

Controls for AI-assisted development: mandatory reading, gates, Bussola, trade-off and guardrails.

Control	Purpose	Failure prevented
Mandatory reading	Agent reads canonical dataflow guide before changing app/data code.	Stale context or wrong authority source.
D-030 compliance	Agent respects filesystem-first pattern and validates dataflow changes.	Direct data bypass and DB-first drift.
Roadmap alignment	PR maps to issue, roadmap task or decision context.	Orphan work outside validated scope.
Bussola	Past, Present, Future and Vision are documented.	Missing temporal traceability.
Local/global trade-off	Agent explains local patch vs global/root-cause fix and debt impact.	Quick fix that creates systemic debt.
For Dummies explanation	Agent gives simple human-readable explanation.	Founder/team operability risk.
Test/build evidence	PR includes build/test/guard evidence.	Unverified AI-generated changes.
Breaking-change escalation	Schema/API/version or decision changes require owner approval.	Unreviewed architectural break.
ICP G1 - Issue Alignment	PR should reference issue/task.	Ambiguous scope.
ICP G2 - Bussola Compliance	Blocking temporal traceability gate.	Process theater without substance.
ICP G3 - Roadmap Alignment	Warning if roadmap alignment is weak.	Roadmap drift.
ICP G4 - D-030 Dataflow	Blocking dataflow compliance gate.	Direct app access to forbidden data paths.
ICP G5 - ADV Classification	Anti-patterns require review.	Hidden risky pattern.
ICP G6 - Breaking Changes	Schema/API/version requires owner approval.	Unapproved breaking change.
Conversational guardrail: quickfix	Warns on “just fix” patterns.	Symptom-only patch.
Conversational guardrail: repeated fix	Warns on repeated bug-fix loop.	Rattoppo su rattoppo.
Conversational guardrail: anti-pattern	Blocks bypass/disable-check patterns.	Governance/security bypass.
Conversational guardrail: scope creep	Surfaces “also add” expansions.	Unbounded PR scope.
Conversational guardrail: workaround	Flags hack/workaround requests.	Debt hidden as solution.

Governance rule
This protocol is not a guarantee of correctness. It is a control surface that makes AI-assisted implementation more reviewable.

Appendix J - Dataflow Contract and App Matrix

Declares how app data access, cross-app navigation and source routing should be validated.

Surface / app	Consumes	Produces	Proof / governance relevance
Layer 1 - User Interaction	Dock, button, context menu, keyboard, launcher.	User action events.	Starts workflows but does not own proof.
Layer 2 - Context Propagation	shell.openApp, AppWrapper, useAppContext.	Context keys: sessionId, kernelId, artifactId.	Controls cross-app continuity.

Layer 3 - State Management	useFilesystem, DataStore, React Query cache.	Query/mutation state, invalidations.	Enforces filesystem-first access.
Layer 4 - Backend and Storage	parser-service, MONL, .aosp.txt, Directus cache.	Persisted stripes, cache/index, diagnostics.	Owns durable source/cache separation.
Finder	session_, kernel_, artifact_, atom_, answer_, audit_, lineage_.	None.	Retrieval lens; must not become proof owner.
Data Intake	question_, questionnaire_, session_.	answer_, answer_draft_, atom_, atom_validation_.	D1 proof-sensitive intake.
Data Enrichment	atom_, atom_validation_, answer_, session_.	link_, atom_.	D2/D3 enrichment and relations.
Graph	atom_, atom_validation_, link_, session_.	None.	Relation inspection; projection, not source.
Pipeline Config	fae_, pipeline_config_.	fae_, pipeline_config_.	Bounded orchestration.
Editor	artifact_, atom_, session_.	artifact_.	Artifact editing, not proof by itself.
Shell	session_, atom_, answer_, link_, audit_, validation_, lineage_.	None.	Query/read surface.
Settings	settings_, user_, provider_.	settings_.	Capabilities, provider and configuration control.
Inspector	session_, atom_, link_, kernel_, flags, stores, schema.	None.	Object inspection, source/proof state visibility.
iKant	session_, atom_, validation_, answer_, kernel_.	None.	Governance-aware assistant surface.
Artifact Workstation	artifact_, template_, session_, kernel_.	artifact_.	Artifact boundary and reviewable outputs.
Observatory	WebSocket event stream, E2E test results.	Diagnostic projections.	Failure/proof-state visibility.
Kernel Browser	kernel_, session_.	None.	Compact synthesis inspection.
Error Dashboard	AOSPErrorEnvelope / unified error store.	Diagnostic projections.	Error Truth surface.
D-030 rule	All persisted, replayable, user-owned data should go through useFilesystem.	Stripe emission and cache updates.	Blocks DB-first app drift.
Auto-source routing	Explicit source, app readSourcePolicy, prefix heuristic.	Canonical/session source selection.	Prevents wrong-source reads.

Dataflow rule
A new app is not complete until its consumes, produces, cross-app navigation and source policy are declared and validated.

Appendix K - Documentation Governance and Generated Control Plane

Documentation is treated as synchronized infrastructure, not ungoverned prose.

Tool / surface	Function	Failure prevented
validate-docs	Checks required docs, metadata, internal links, naming conventions, duplicate scripts, temporary directories and reachability.	Broken documentation map and stale links.
validate-integration	Checks synchronization across roadmap, blueprints, development line, registry, orphan detection and agent instruction integration.	Local changes disconnected from roadmap/system.
update-registry	Scans active documents and updates registry metadata, relationships and status.	Documentation registry drift.
archive-obsolete	Moves obsolete documents to archive while preserving replacement mappings and traceability.	Stale docs masquerading as authority.
deep-clean-docs	Performs forensic markdown cleanup: frontmatter, stale docs, duplicates, dead links, placeholders and quarantine.	Accumulated documentation debt.

transclude	Synchronizes markdown blocks through BLOCK/TRANSCLUDE syntax with check/update/dry-run mode.	Manual source-to-sink drift.
strategic-coherence	Validates whether local changes are authorized by active task, phase, wave focus, roadmap and decision log.	Unauthorized work and local/global incoherence.
Authority Matrix	Declares SSOT, Derived, Advisory and Archaeological surfaces.	AI agent follows stale or derived doc as source truth.
App Support Tier Matrix	Declares tier-0, tier-1, tier-2 and tier-3 app maturity.	Experimental surfaces sold as core.
EQL Capability Matrix	Declares which surfaces support which EQL capabilities.	UI suggests unsupported proof query.
Certified Examples	Stores accepted examples for regression and documentation.	Query DSL behavior drifts silently.
Knowledge Center	Generated or curated navigation over docs and proof surfaces.	Lost discoverability.
Classification Registry	Classifies entities as canonical, active, dormant, archaeological or dead.	Dormant/dead surfaces become invisible.
Generated docs rule	Generated docs are read-only derived surfaces and should be regenerated from source artifacts.	Manual edits to generated truth.

Documentation rule
If documentation influences implementation or governance, it must declare whether it is source, derived, advisory or archaeological.

Appendix L - Topology and Reachability System

Validation of apps, routes, calls, E2E journeys, hooks and repository reachability.

Tool / concept	What it does	Drift detected
topology:validate	Validates topology contract against expected structure.	Declared topology contract broken.
topology:lint	Enforces topology rules and canonical constraints.	Rule violation in topology.
topology:sync	Synchronizes topology contract from code.	Generated contract stale against implementation.
scan-backend	Discovers backend services/routes.	Undocumented backend route or service.
validate-e2e	Cross-validates E2E topology and journeys.	E2E map does not match surfaces.
scan-actual-calls	Scans actual API calls in the codebase.	Frontend/backend call mismatch.
ast-scanner	Scans frontend apps for hooks and usage patterns.	Hook usage invisible to topology.
scan-intents	Maps global intents across frontend and backend.	Global user intent not covered by routes/tests.
compare	Compares topology versions.	Unexplained topology drift.
Canonical	Foundation document or surface, actively maintained and cross-referenced.	None; expected authority.
Active	Currently used in runtime, CI/CD or development workflow.	Active but undocumented surface.
Dormant	Present but not wired; has activation conditions.	Dormant surface treated as active.
Archaeological	Historical reference only, preserved for context.	Historical doc used as current truth.
Dead	No references or value; candidate for removal.	Dead surface accumulating debt.
Reachability test	Checks whether entity is referenced by runtime	Invisible orphan artifact.

	code, docs or CI/CD.	
Effect test	Checks whether entity produces runtime, build, test, CI or deployment effect.	Decorative no-effect artifact.
Coherence test	Checks whether content reflects current codebase state.	Stale documentation or config.
Informational value test	Checks unique decisions, rationale or history.	Low-value clutter.
Cross-reference test	Checks canonical/archeological links.	Unlinked authority or archive.

Topology moat
A-OSP should not allow hidden app surfaces, undocumented routes, orphan E2E tests or stale architecture diagrams to accumulate without classification.

Appendix M - Security and Production Readiness

Development adequacy and production readiness must be separated explicitly.

Area	Current / target posture	Validation KPI
Authentication	Token-based auth exists; production should harden toward httpOnly cookies and session controls.	auth_hardening_completion
Protected routes	Route guards check authentication and show loading/redirect states.	protected_route_coverage
Input validation	Basic login/email/required-field validation; server-side validation remains required.	input_validation_coverage
RBAC	Backend/auth layer enforces role and permission boundaries.	role_permission_test_pass_rate
CORS	Development CORS is limited to localhost; production requires strict allowed origins.	cors_policy_validation
Secrets	Production requires secrets management and no committed .env/secrets.	secret_scan_pass_rate
HTTPS	Production requires HTTPS and valid certificates.	https_enforced_rate
Security headers	CSP, X-Frame-Options and related headers should be implemented before production.	security_header_coverage
Rate limiting	Login/API endpoints require rate limiting and brute-force protection.	rate_limit_coverage
CSRF	Production cookie/session architecture requires CSRF protection.	csrf_test_pass_rate
Audit logging	Audit logs should track auth attempts, data modifications and privileged operations.	audit_log_completeness
Dependency scanning	npm audit and Trivy scan dependencies/filesystem/images where configured.	high_critical_vulnerability_count
Provider exposure	Each model call should classify payload, allowed refs, provider and retention assumption.	provider_payload_classification_coverage
Tenant/session isolation	Role, ownership and tenant/session boundaries should be tested.	tenant_isolation_test_pass_rate
Backup/restore	Files, cache and proof objects should be restorable from source artifacts.	backup_restore_success_rate
Production blocker list	Open production blockers should remain explicit and tracked.	production_security_blocker_count

Security maturity rule
A-OSP should not claim production readiness until production blockers are closed and validated by explicit metrics.

Appendix N - Local / Global Proof Journey Example

One end-to-end example connecting user input, local objects, proof, artifact, governance and operational evidence.

Step	A-OSP operation	Objects / evidence produced	Proof or governance meaning
1	Capture supplier payment-control answer.	answer_001	Human input becomes local source object.
2	Atomic D1 extraction proposes control fact and evidence gap.	atom_001, atom_002	AI output is candidate structure, not proof.
3	Persist atoms immediately as pending.	atom_ status: pending	Persist-first lifecycle improves crash recovery.
4	Human validates or modifies atoms.	atom_validation_001, atom_validation_002	Human gate before downstream reliance.
5	D1 proof transition persists, reads back and compares.	D1Receipt	D1 becomes proof-sensitive only with readback/receipt.
6	EQL scoped retrieval selects relevant objects.	query_scope, query_hash, result_hash, EQLQueryReceipt	Retrieval is bounded and reproducible.
7	FAE bounded orchestration selects focus, attention and execution.	fae_ profile / call envelope	Model call is phase-scoped and constrained.
8	D5 kernel compactly synthesizes the state.	kernel_001	Reusable epistemic compression with dependencies.
9	Artifact Workstation assembles reviewable artifact.	artifact_001	Export is not proof unless sources/gaps/status are exposed.
10	Artifact boundary exposes unsupported claims and missing evidence.	ArtifactReceipt / ArtifactWitness target	Review sees dependencies and proof boundary.
11	GovernanceRun records reviewer decision.	GovernanceRun	Human decision over proof objects and follow-up actions.
12	Local kernel is proposed for team/enterprise promotion.	promotion candidate / diff	Local truth does not automatically become enterprise truth.
13	Review/diff/conflict handling accepts or rejects promotion.	promotion receipt / rollback path	Enterprise canonical state is governed.
14	If a failure occurs, Error Truth emits canonical envelope.	AOSPErrorEnvelope	Failure becomes diagnosable: root cause, state, next action.
15	If code changed, PR guard evidence is attached.	issue link, Bussola, tests, guard report	Repository development remains inspectable.
16	End-to-end result.	object chain + proof/diagnostic/governance evidence	A human answer becomes a typed, queryable, reviewable, diagnosable and governable chain.

End-to-end thesis
A-OSP transforms a human answer into a typed, validated, queryable, reviewable, diagnosable and governable chain of objects.

Appendix O - Object Mapping, Concept Coverage and Review Standards

This appendix extracts the principal objects, controls and concepts described in the whitepaper and maps them to their definition, engineering section, maturity status, validation metric and review audience. It also provides a legacy-to-current concept coverage matrix so concepts introduced in earlier DOCX versions remain traceable in the current master.

Object / concept	Meaning	Primary section	Maturity	Primary KPI	Review audience
.aosp.txt stripe	Durable text-backed source event/object.	Ch.7-9	AS-IS/HARDENING	stripe_parse_success_rate	CTO/Gov
Stripe Schema Registry	Operational ontology for prefixes and schemas.	Ch.8	AS-IS/HARDENING	schema_validation_pass_rate	CTO
Dynamic schema	Minimal inferred schema for new prefixes.	Ch.8	AS-IS/RISK-CONTROL	unknown_prefix_rate	CTO/Gov
answer_	Captured human answer.	Ch.9/Annex N	AS-IS	answer_persistence_rate	Gov
atom_	Extracted fact, evidence item or gap.	Ch.9/Annex N	AS-IS/HARDENING	atom_validation_rate	Gov/CTO
evidence_gap	First-class absence-of-evidence object.	Ch.9/23	AS-IS/TARGET	missing_evidence_detection_rate	Governance
enrichment_	Context/reference enrichment.	Ch.9	TARGET/HARDENING	enrichment_lineage_coverage	CTO
link_	Relationship between objects.	Ch.9	AS-IS/HARDENING	relation_density	CTO
score_/validation_	Evaluation or human validation state.	Ch.9/14	HARDENING	validation_completion_rate	Governance
kernel_	Compact D5 synthesis with dependencies.	Ch.9/23	HARDENING/TARGET	kernel_dependency_completeness	Investor/CTO
EQL	Epistemic Query Language for scoped retrieval.	Ch.11	CONTRACT/HARDENING	eql_query_receipt_coverage	CTO
FAE profile	Focus-Attention-Execution orchestration object.	Ch.12	HARDENING	fae_output_schema_pass_rate	CTO
MONL operator	Parsed-stripe operator layer.	Ch.13	CONDITIONAL/TARGET	monl_diff_success_rate	CTO
D1Receipt	Receipt for proof-sensitive intake.	Ch.14	HARDENING	D1Receipt_coverage	Governance
EQLQueryReceipt	Receipt metadata for proof query.	Ch.11/14	TARGET	query_hash_result_hash_coverage	CTO/Gov
PhaseReceipt/PipelineReceipt	Receipts for D2-D5 and pipeline lineage.	Ch.14/25	TARGET	pipeline_receipt_coverage	CTO
ArtifactReceipt	Proof-boundary receipt for artifact sealing/export.	Ch.15	HARDENING	artifact_receipt_coverage	Governance
ArtifactWitness	Dependency map of sources, gaps, unsupported claims.	Ch.15	TARGET	artifact_witness_coverage	Gov
GovernanceRun	Human review context and decision object.	Ch.15/23	TARGET	governance_run_coverage	Gov/Government
Proof Spine	Target composition layer across proof objects.	Ch.14	TARGET	proof_bundle_completeness	Gov
AOSPErrorEnvelope	Canonical forensic error object.	Ch.17	CONTRACT/TARGET	error_envelope_coverage	CTO/Gov
Capability state	Runtime availability/degradation grammar.	Ch.18	CONTRACT/TARGET	capability_state_coverage	CTO
Projection	UI/export read-shape over canonical identity.	Ch.18	CONTRACT/TARGET	projection_consistency_rate	Gov
Authority Matrix	Repository authority classification map.	Ch.19	AS-IS	authority_matrix_check_pass_rate	Governance
App Support Tier	Product maturity map for app lenses.	Ch.16	AS-IS	tier0_e2e_coverage	CTO
OperationalMoatBundle	Metrics bundle for CI/test/guard/script surfaces.	Ch.26/Annex E-F	TARGET	guard_pass_rate	Investor/CTO
D1ProofAppendService	Backend producer of proof-grade D1 append, receipt and readback.	Ch.25 / Appendix P	MISSING / HARDENING	d1_proof_append_pass_rate	CTO/Gov
d1_receipt_	Persisted D1 proof receipt stripe.	Ch.14 / Appendix P	MISSING / HARDENING	d1_receipt_persistence_rate	CTO/Gov
EQLRunResult	Unified envelope for parse/capability/execution/result/proof state.	Ch.11 / Appendix P	MISSING / HARDENING	eql_run_result_coverage	CTO
EQLQueryReceipt	Idempotent proof/canonical query receipt with query/result hashes.	Ch.11 / Appendix P	MISSING / HARDENING	eql_query_receipt_coverage	CTO/Gov
C1 convergence checkpoint	Final D1/EQL proof-seam report and negative-test artifact gate.	Ch.25 / Appendix P	MISSING / TARGET	c1_negative_test_pass_rate	CTO/Gov/Investor

Appendix P - D1/EQL Proof-Grade Engineering Attestation

P.1 Purpose and claim boundary

This appendix compresses the D1/EQL proof-grade engineering attestation into the whitepaper. It is not a certification. It is an engineering-state attestation anchored to a specific repository commit, internal normative documents and the open issue chain.

Reference state:

Repository: Luke883i/aosp1
Reference commit: aa68898cd1b882549e8b354aad894dc86629566e
SSOT version: 1.0.0-mvp-rc3
Phase: Phase 10 - Hardening & MVP, in progress
Attestation date: 2026-05-26

Canonical status vocabulary:

LANDED
RUNTIME-WIRED
RUNTIME-WIRED-BUT-UNTESTED
CONTRACT-ONLY
SCAFFOLD-NOT-WIRED
MISSING
DEPRECATED

Evaluation rule:

No layer is considered PASS if one proof-critical subcomponent is MISSING or CONTRACT-ONLY.
No non-false-green assertion is accepted without a guardian test.
Each gap must be OWNED-BY-ISSUE or marked UNCOVERED.

P.2 Executive verdict

D1+EQL proof-grade is not reached at the reference commit.

However, the gap is fully mapped: 14/14 known D1/EQL proof-grade gaps are owned by open issues.

D1/EQL proof-grade: NOT REACHED
Open issue coverage: 14/14 mapped
Uncovered gaps: 0
Main risk: foundations exist, but the end-to-end proof seam is not closed

This is the correct whitepaper posture

A-OSP may claim:
landed D1/EQL foundations;
runtime-wired DataIntake UI honesty;
landed EQL L0-L2 parser/capability/virtual-view foundations;
issue-owned plan to close D1/EQL proof-grade.

A-OSP should not yet claim:
D1/EQL proof-grade completion;
real backend D1 proof PASS production;
EQL proof receipts;
guaranteed Finder/Shell proof handoff;
D2-D5 consumption only from D1Receipt-backed proof state.

P.3 Layer-state summary

Layer	Current coded state	M1 blocker	Owner
L0 proof contracts shared	LANDED	No	-
L1 D1 contracts shared	LANDED	No	-
L1.UI DataIntake honesty	RUNTIME-WIRED	No	-
L1.UI.SHELL enterprise layout	SCAFFOLD-NOT-WIRED	Yes	#3462
L1.BACKEND D1 proof producer	MISSING	Yes	#3462 Addendum P0
GET /api/v1/d1/capability	RUNTIME-WIRED	No	-
D1 to D2 hard-gate on receipt	MISSING	Yes	#3462
L2 pipeline contracts	LANDED / contract-only	No for M1	#3471 M2
EQL L0 capability matrix/examples	LANDED	No	-
EQL L1 parser + AST	LANDED	No	-
EQL L2 virtual views/tokenizer	LANDED	No	-
EQL L3 EQLRunResult / EQLQueryReceipt / service	MISSING	Yes	#3425
EQL L4 unified route POST /api/eql/run	MISSING	Yes	#3426
EQL L5 canonical hash + idempotent receipt	MISSING	Yes	#3427
EQL L6 proof scripts + CI workflow	MISSING	Yes	#3430
EQL proof UX components	MISSING	Post-M1 / demo-relevant	#3428
C1 convergence artifacts	MISSING	Yes	#3488

P.4 What is already strong

The repository already contains a substantial D1/EQL proof foundation.

```
Landed or runtime-wired foundations:
 shared proof envelope/context/transaction/capability/problem-detail contracts
 D1Receipt schema
 D1ProofMode
 D1 receipt green predicate
 CanonicalAppendSDK interface
 D1 UI proof view model
 CompletionModal D2 gate
 D1 truthful-completion E2E guard
 EQL capability matrix
 certified EQL examples
 EQL strict parser and AST
 EQL virtual views
 EQL tokenizer v1
 pipeline receipt contracts
```

Most important positive finding:

DataIntake UI is currently fail-closed.
It cannot honestly show D1 proof green unless D1 proof PASS is represented through the D1 proof view model.

This already prevents the historical false-green class:

```
questionnaire complete != proof complete
validated=true != proof
fallback/demo/local != proof
generic append success != proof PASS
```

P.5 Main missing proof seam

The central missing element is the D1 proof producer.

Required but missing:

```
POST /api/v1/proof/d1/append
D1ProofAppendService
persisted d1_receipt_
read-after-write barrier
D1Receipt PASS with readback
proofSessionContext containing sessionId + receipt id + hash
frontend proofAppendD1 / CanonicalAppendSDK runtime use
useD1ProofStatus reading real d1_receipt_
D2 handoff blocked without receipt PASS
```

Separation rule:

```
Generic append may create canonical data.
Generic append must not create proof PASS.
```

Proof PASS requires:

```
 D1 proof append path
 persisted receipt
 deterministic readback
 explicit scope
```

P.6 D1 probe matrix

Probe	Question	Current verdict	MVP required
S1	Is D1Receipt persisted as d1_receipt_*?	NO	YES
S2	Is CanonicalAppendSDK the only proof path?	NO / CONTRACT-ONLY	YES
S3	Does strict mode block fallback/mock/demo?	PARTIAL	YES
S4	Does useD1ProofStatus read persisted receipt?	NO	YES
S5	Does CompletionModal require D1 PASS + readback?	YES, UI only	YES
S6	Is D2 handoff hard-gated on receipt?	NO	YES
S7	Is d1_receipt_* queryable/used by EQL?	PARTIAL	YES

Summary: 3 PASS / 4 NO-or-PARTIAL across MVP-required probes.

P.7 EQL current state

EQL has strong foundations but is not yet proof-grade.

Already landed:

```
Capability matrix
Certified examples
Strict parser
Typed AST
Unsupported-feature rejection path
Virtual views:
 _stripe_
 _receipt_
 _phase_
 _lineage_
 _text_index_
Tokenizer v1
Text index
Executor over stripe corpus
```

Missing proof-grade layer:

```
EQLRunResult
EQLQueryReceipt
QueryScope
EQLMode
canonicalQuery
astHash
queryHash
resultHash
idempotent receipt store
POST /api/eql/run
proof-mode explicit-scope enforcement
EQL proof CI workflow
```

EQL proof must not mean "the query returned rows". It must mean:

```
parse PASS
capability PASS
scope explicit
execution PASS
result classified
hashes emitted
receipt emitted when proof/canonical mode requires it
```

P.8 EQL UX state

EQL UX is still missing as a unified cross-surface proof layer.

Required components:

```
EQLStatusCard
EQLScopeBadge
EQLProofBadge
EQLExamplePicker
EQLUnsupportedFeatureHelp
EQLZeroResultExplanation
EQLModeChip
useEQLRun
useEQLValidate
```

Required UX distinction:

VALID_ZERO:
 query executed correctly and returned zero rows

FAIL_EMPTY:
 proof-sensitive query expected rows but returned none

NOT_EXECUTED:
 parse/capability/execution failed

External-demo rule:

Even if full EQL UX remains post-M1, no external proof-grade demo should present EQL as proof-ready without a minimal visible proof-state surface.

P.9 Issue-chain coverage

Gap	Missing / partial item	Owner	M1 blocker
G1	POST /api/v1/proof/d1/append + D1ProofAppendService	#3462	Yes
G2	persisted d1_receipt_ + read-after-write barrier	#3462	Yes
G3	runtime CanonicalAppendSDK / proofAppendD1	#3462	Yes
G4	useD1ProofStatus reads real d1_receipt_	#3462	Yes
G5	DataIntakeShell wired into DataIntakeApp.tsx	#3462	Yes
G6	Finder/Shell handoff with explicit sessionId	#3462	Yes
G7	strict preflight non-dismissible	#3462	Yes
G8	D2 handoff hard-gated on D1Receipt	#3462 / #3471	Yes
G9	EQLRunResult, EQLQueryReceipt, eqlQueryService	#3425	Yes
G10	unified route POST /api/eql/run	#3426	Yes
G11	hash canonicalization + idempotent EQL receipt	#3427	Yes
G12	EQL proof scripts + eql-proof.yml	#3430	Yes
G13	EQL anti-goals enforcement	#3431	Yes / hardening
G14	C1 checkpoint artifacts + convergence report	#3488	Yes

P.10 Adequacy of the planned issue chain

The planned issue chain is adequate because it attacks the proof-critical seams directly:

#3462 closes the D1 producer, receipt, readback, proofSessionContext and handoff gap.
#3425 defines the EQL result/receipt envelope.
#3426 removes parallel EQL execution paths.
#3427 makes EQL proof reproducible through canonicalization and hashes.
#3430 turns EQL proof into scripts and CI evidence.
#3431 prevents unsupported or fake EQL proof surfaces.
#3488 forces convergence through negative tests and report artifacts.

Adequacy verdict:

Backend/proof adequacy: high
D1 UI honesty adequacy: high
EQL engine adequacy after #3425-#3430: high
Roadmap coverage: high
External-demo UX adequacy: medium unless minimal EQL proof-state UI is pulled forward

Required adjustment:

A minimal EQL proof UX surface should be treated as a demo/readiness gate, even if full EQL UX remains M2.

P.11 Latent sub-gaps

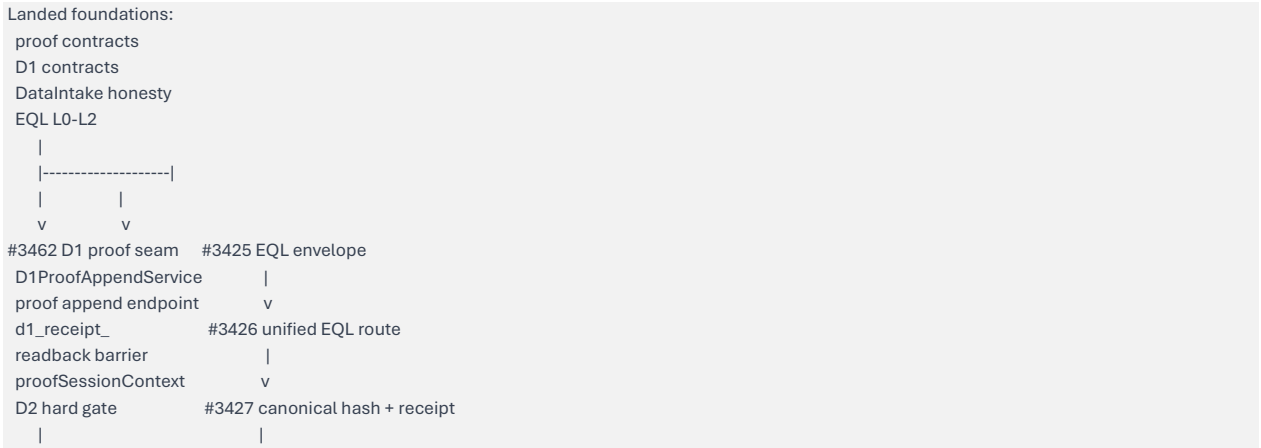
Two latent sub-gaps should remain explicit.

SG-1:
d1_receipt_ is not recognized by EQL _receipt_ virtual view.
When d1_receipt_ is introduced, it must be added to receipt virtual-view prefixes.

SG-2:
eql_query_ is recognized as a receipt-like prefix, but no runtime producer exists yet.
It becomes meaningful only after EQLQueryReceipt and EQLReceiptStore land.

These should be closed inside the existing issue chain rather than creating a parallel roadmap.

P.12 Closure DAG



```
|                        v
|                #3430 proof scripts + CI
|                        |
|                        v
|                 #3431 anti-goals enforcement
|                        |
|--------------------|
      v
#3488 C1 convergence checkpoint
      v
M1 close: proof-grade D1/EQL attested
```

P.13 Required C1 artifacts

C1 should not be considered closed without these artifacts:

```
docs/convergence/C1_D1_EQL_PROOF_SEAM.md
docs/generated/convergence-c1-report.md
artifacts/convergence/C1_D1_EQL_LEGACY_SCAN.json
artifacts/convergence/C1_FALSE_GREEN_RESULTS.json
scripts/convergence/c1-d1-eql-legacy-scan.mjs
scripts/convergence/c1-d1-eql-false-green-check.mjs
```

Required negative tests:

```
generic append success without D1Receipt => FAIL
questionnaire complete without D1Receipt => FAIL
D1Receipt without readback => FAIL
stale D1Receipt => FAIL
fallback/demo/local D1 => no proof PASS
D2 handoff without D1Receipt PASS => FAIL
EQL proof query without scope => FAIL
global EQL proof query => FAIL
content-only EQL proof query => FAIL
EQL success without EQLQueryReceipt => no proof
```

P.14 Final appendix verdict

D1/EQL proof-grade is not complete today.

The foundation is substantial:

```
contracts landed,
DataIntake honesty wired,
EQL parser/capability/virtual views landed.
```

The missing seam is precise:

```
D1 proof producer,
persisted receipt,
readback barrier,
EQL proof envelope,
EQL receipt,
unified EQL route,
hash canonicalization,
C1 negative-test artifacts.
```

The plan provided that:

```
#3462 and #3425-#3430 land before any proof-grade claim;
#3488 becomes the external attestation gate;
a minimal EQL proof UX surface is pulled forward for external demonstration;
SG-1 and SG-2 are closed inside the existing issue chain.
```